

SA.: Phase 4 Code Challenge: Superheroes

- Due Aug 10, 2025 by 11:59pm
- Points 15
- Submitting a website url
- Available Aug 6, 2025 at 12am - Aug 29, 2025 at 11:59pm

This assignment was locked Aug 29, 2025 at 11:59pm.

Instructions

For this assessment, you'll be working on an API for tracking heroes and their superpowers.

Part of what this code challenge is testing is your ability to follow given instructions. While you will have a significant amount of freedom in how you implement the features, be sure to carefully read the directions for setting up the application

Setup

1. Create a new **PRIVATE** repository in this GitHub Organization ([Moringa-SDF-PT10](https://github.com/Moringa-SDF-PT10) (<https://github.com/Moringa-SDF-PT10>) (<https://github.com/Moringa-SDF-PT07>)). Ensure your repository has a name in the following format; *superheroes-firstname-lastname* (Example: **superheroes-jane-doe**). *Take note, any other repository name will **NOT** be graded.*
2. You have been provided a [Postman collection](https://moringa.instructure.com/courses/1054/files/558299?wrap=1) (<https://moringa.instructure.com/courses/1054/files/558299?wrap=1>)  (https://moringa.instructure.com/courses/1054/files/558299/download?download_frd=1). This collection contains all the endpoints that you are required to create with this API. You can download and import it into your Postman application to test that your app works correctly.

Select `Upload Files`, navigate to this repo folder, and select `challenge-2-superheroes.postman\_collection.json` as the file to import.

Before you submit! Save and run your code to verify that it works as you expect.

You **MUST** have a well-written **README** in your repository. Ensure your markdown renders correctly before submission. You can use [Visual Studio Code Markdown preview](https://code.visualstudio.com/docs/languages/markdown#_markdown-preview) (https://code.visualstudio.com/docs/languages/markdown#_markdown-preview) to see how it would appear on your **GitHub** repository.

Resources

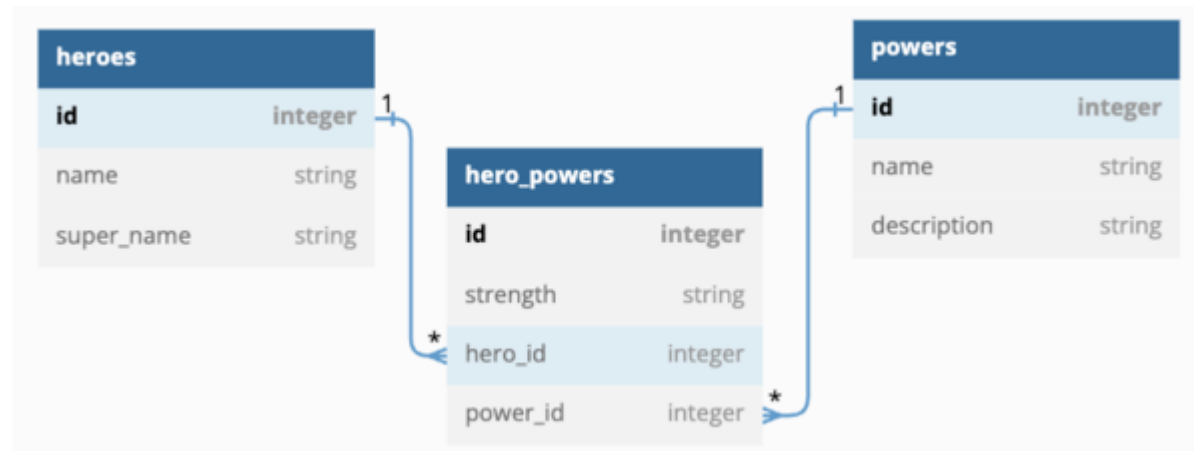
- [How to write a good README. \(https://www.freecodecamp.org/news/how-to-write-a-good-readme-file/\)](https://www.freecodecamp.org/news/how-to-write-a-good-readme-file/)

Deliverables

Your job is to build out the Flask API to add the functionality described in the deliverables below.

Models

You will implement an API for the following data model:



Now you can implement the relationships as shown in the ER Diagram:

- A **Hero** has many **Power**'s through **HeroPower**
- A **Power** has many **Hero**'s through **HeroPower**
- A **HeroPower** belongs to a **Hero** and belongs to a **Power**

Since a `HeroPower` belongs to a `Hero` and a `Power`, configure the model to cascade deletes.

Set serialization rules to limit the recursion depth.

Run the migrations and seed the database

If you aren't able to get the provided [seed file \(https://moringa.instructure.com/courses/1054/files/558300?wrap=1\)](https://moringa.instructure.com/courses/1054/files/558300?wrap=1)  [working](https://moringa.instructure.com/courses/1054/files/558300/download?download_frd=1), you are welcome to generate your own seed data to test the application.

Validations

Add validations to the `HeroPower` model:

- `strength` must be one of the following values: **'Strong', 'Weak', 'Average'**

Add validations to the `Power` model:

- `description` must be present and at least 20 characters long

Routes

Set up the following routes. Make sure to return JSON data in the format specified along with the appropriate HTTP verb.

Recall you can specify fields to include or exclude when serializing a model instance to a dictionary using `to_dict()` (don't forget the comma if specifying a single field).

a. GET /heroes

Return JSON data in the format below:

```
[
  {
    "id": 1,
    "name": "Kamala Khan",
    "super_name": "Ms. Marvel"
  },
  {
    "id": 2,
    "name": "Doreen Green",
    "super_name": "Squirrel Girl"
  },
  {
    "id": 3,
    "name": "Gwen Stacy",
    "super_name": "Spider-Gwen"
  },
  {
    "id": 4,
    "name": "Janet Van Dyne",
    "super_name": "The Wasp"
  },
  {
    "id": 5,
    "name": "Wanda Maximoff",
    "super_name": "Scarlet Witch"
  },
  {
    "id": 6,
    "name": "Carol Danvers",
    "super_name": "Captain Marvel"
  },
]
```

```
{
  "id": 7,
  "name": "Jean Grey",
  "super_name": "Dark Phoenix"
},
{
  "id": 8,
  "name": "Ororo Munroe",
  "super_name": "Storm"
},
{
  "id": 9,
  "name": "Kitty Pryde",
  "super_name": "Shadowcat"
},
{
  "id": 10,
  "name": "Elektra Natchios",
  "super_name": "Elektra"
}
]
```

b. GET /heroes/:id

If the `Hero` exists, return JSON data in the format below:

```
{
  "id": 1,
  "name": "Kamala Khan",
  "super_name": "Ms. Marvel",
  "hero_powers": [
    {
      "hero_id": 1,
      "id": 1,
      "power": {
        "description": "gives the wielder the ability to fly through the skies at supersonic speed",

```

```
        "id": 2,  
        "name": "flight"  
      },  
      "power_id": 2,  
      "strength": "Strong"  
    }  
  ]  
}
```

If the `Hero` does not exist, return the following JSON data, along with the appropriate HTTP status code:

```
{  
  "error": "Hero not found"  
}
```

c. GET /powers

Return JSON data in the format below:

```
[  
  {  
    "description": "gives the wielder super-human strengths",  
    "id": 1,  
    "name": "super strength"  
  },  
  {  
    "description": "gives the wielder the ability to fly through the skies at supersonic speed",  
    "id": 2,  
    "name": "flight"  
  },  
  {  
    "description": "allows the wielder to use her senses at a super-human level",  
    "id": 3,  
    "name": "super human senses"  
  },  
]
```

```
"description": "can stretch the human body to extreme lengths",  
"id": 4,  
"name": "elasticity"  
}  
]
```

d. GET /powers/:id

If the `Power` exists, return JSON data in the format below:

```
{  
  "description": "gives the wielder super-human strengths",  
  "id": 1,  
  "name": "super strength"  
}
```

If the `Power` does not exist, return the following JSON data, along with the appropriate HTTP status code:

```
{  
  "error": "Power not found"  
}
```

e. PATCH /powers/:id

This route should update an existing `Power`. It should accept an object with the following properties in the body of the request:

```
{  
  "description": "Valid Updated Description"  
}
```

If the `Power` exists and is updated successfully (passes validations), update its description and return JSON data in the format below:

```
{  
  "description": "Valid Updated Description",  
  "id": 1,  
}
```

```
"name": "super strength"
}
```

If the `Power` does not exist, return the following JSON data, along with the appropriate HTTP status code:

```
{
  "error": "Power not found"
}
```

If the `Power` is **not** updated successfully (does not pass validations), return the following JSON data, along with the appropriate HTTP status code:

```
{
  "errors": ["validation errors"]
}
```

f. POST /hero_powers

This route should create a new `HeroPower` that is associated with an existing `Power` and `Hero`. It should accept an object with the following properties in the body of the request:

```
{
  "strength": "Average",
  "power_id": 1,
  "hero_id": 3
}
```

If the `HeroPower` is created successfully, send back a response with the data related to the new `HeroPower`:

```
{
  "id": 11,
  "hero_id": 3,
  "power_id": 1,
  "strength": "Average",
  "hero": {
    "id": 3,
```



```
    "name": "Gwen Stacy",  
    "super_name": "Spider-Gwen"  
  },  
  "power": {  
    "description": "gives the wielder super-human strengths",  
    "id": 1,  
    "name": "super strength"  
  }  
}
```

If the `HeroPower` is **not** created successfully, return the following JSON data, along with the appropriate HTTP status code:

```
{  
  "errors": ["validation errors"]  
}
```